

Reviewer Pack — Minimal Rank of Tropicalized Group Representations Bounds XO...

Entry 5564f7b774e2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 17:48:01 UTC

Minimal Rank of Tropicalized Group Representations Bounds XOR-AND Tree Width

Entry ID: 5564f7b774e2 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 17:48:01 UTC

1. Conjecture

Field A (mathematical branch): Tropical Representation Theory **Field B** (complexity object): Complexity Theory: XOR-AND Tree Width

Statement:

[‘For any given group G and its representation V , the tropicalized version of V , denoted as V_{trop} , has a minimal rank $r(V_{\text{trop}})$ such that any XOR-AND tree representing the same group action on a set of size n requires at least $r(V_{\text{trop}})$ levels.’, ‘Specifically, for all instances with $|G| \leq 40$ and $|V| \leq 40$, if the XOR-AND tree width is w , then $r(V_{\text{trop}}) \leq \log_2(w + 1)$.’, ‘Moreover, there exists a constructive mapping from an instance of group action representation to V_{trop} in at most 50 lines of Python.’]

Rationale (proposer’s reasoning):

[‘Tropicalization is a technique that has been applied in various branches of mathematics and can potentially expose hidden structures in complexity theory objects. By using tropicalized group representations, we might reveal new properties of XOR-AND trees that are not apparent from classical representation theory.’, ‘The conjecture leverages the connections between algebraic objects (group representations) and computational complexity (XOR-AND tree width), which could potentially lead to novel algorithms or lower bounds in complexity theory.’, ‘Tropicalization is computationally feasible, as it involves basic operations over the tropical semiring that can be implemented efficiently.’]

Taxonomy category: TROPICAL_GROUP_REPRESENTATION (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f00adddb57686369

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the mean correlation coefficient between $r(V_{\text{trop}})$ and $\log_2(w + 1)$ across 30 random seeds is greater than or equal to 0.8, with no seed producing a metric value > 10 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropical representation theory AND XOR-AND tree width - minimal rank tropicalized group representations AND bounds on tree width - constructive mapping from group representation to tropicalization AND complexity theory

Top relevant hits considered: - [<http://arxiv.org/abs/1207.2443v2>] Tropical Teichmüller and Siegel spaces - [<http://arxiv.org/abs/1204.3875v2>] Tropicalizing vs Compactifying the Torelli morphism - [<http://arxiv.org/abs/1204.6154v2>] Local Tropicalization - [<http://arxiv.org/abs/1405.2700v1>] Zero Excess and Minimal Length in Finite Coxeter Groups - [<http://arxiv.org/abs/1908.06409v1>] Schur multipliers of special p-groups of rank 2 - [<http://arxiv.org/abs/hep-ph/0610012v1>] Tevatron-for-LHC Report of the QCD Working Group - [<http://arxiv.org/abs/1110.6618v2>] Brauer relations in finite groups II - quasi-elementary groups of order p^{aq} - [<http://arxiv.org/abs/1101.2456v3>] Polynomial representations and categorifications of Fock Space

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Generate a random group G and its representation V with |G| ≤ 40 and |V| ≤ 40
    n = random.randint(5, 30)
    G = list(range(n))
    V = [[random.random() for _ in range(n)] for _ in range(n)]

    # Construct the tropicalized version of V
    def tropicalize(V):
        n = len(V)
        V_trop = [[max(a, b) for a, b in zip(row1, row2)] for row1, row2 in V]
        return V_trop

    V_trop = tropicalize(V)

    # Construct an XOR-AND tree representing the group action and measure its width w
    def construct_xor_and_tree(G):
```

```

    if len(G) == 1:
        return 1
    else:
        mid = len(G) // 2
        left_tree = construct_xor_and_tree(G[:mid])
        right_tree = construct_xor_and_tree(G[mid:])
        return max(left_tree, right_tree) + 1

w = construct_xor_and_tree(G)

# Calculate the minimal rank r(V_trop)
def matrix_rank(matrix):
    m, n = len(matrix), len(matrix[0])
    rank = 0
    for i in range(m):
        if any(matrix[i][j] != 0 for j in range(n)):
            rank += 1
            for j in range(n):
                matrix[i][j] /= matrix[i][j]
            for k in range(m):
                if k != i and any(matrix[k][j] != 0 for j in range(n)):
                    for j in range(n):
                        matrix[k][j] -= matrix[k][i] * matrix[i][j]
    return rank

r_V_trop = matrix_rank(V_trop)

# Calculate the correlation between r(V_trop) and log_2(w + 1)
if w == 0:
    correlation = None
else:
    correlation = r_V_trop / math.log2(w + 1)

return {
    "metric_name": "correlation",
    "metric_value": correlation,
    "instances_tested": 1,
    "conjecture_holds": correlation is not None and correlation <= 10,
    "counterexample": "" if correlation is not None else "mapping_undefined"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result["metric_value"])

    mean = sum(results) / len(results)
    std = math.sqrt(sum((x - mean) ** 2 for x in results) / len(results))
    support_fraction = sum(1 for r in results if r is not None and r <= 10) / len(results)

    if all(r is not None and r <= 10 for r in results):

```

```

    print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
elif any(r is not None and r > 10 for r in results):
    first_failing_seed = next(i for i, r in enumerate(results) if r is not None and r > 10)
    print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
else:
    print(f"RESULT: INCONCLUSIVE reason=mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bf5e5260.py", line 83, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bf5e5260.py", line 32, in run_trial
    V_trop = tropicalize(V)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bf5e5260.py", line 29, in tropicalize
    V_trop = [[max(a, b) for a, b in zip(row1, row2)] for row1, row2 in V]
              ^^^^^^^^^^^^^
ValueError: too many values to unpack (expected 2)

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to provide the necessary results for evaluating the conjecture. | next: Investigate and fix the crash in the test code to ensure it can run to completion and produce the required data.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12657
2	preregistration	ollama_remote	glm4:latest	0	0	5694
3	novelty	ollama_remote	glm4:latest	0	0	4555
4	novelty	ollama_remote	glm4:latest	0	0	6232
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12884
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8034

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8148
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9952
9	judge	ollama_remote	glm4:latest	0	0	8140

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 76294 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/5564f7b774e2.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/5564f7b774e2.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/5564f7b774e2.tar.gz (if generated)